Geomatory:

DCC_CodeWave

------conversion----

rad=deg*PI/180.0

deg=rad*180.0/PI

arcM=deg*60.0

deg=arcM/60.0

--------Circle-------

Side lengths: a, b, c

Semiperimeter: p = (a + b + c)/2

Area: A=sqrt(p*(p-a)*(p-b)*(p-c))

Circumradius: R=abc/4A

Inradius: r=A/p, A=rp

Law of sines: sin(alpha)/a = sin(beta)/b = sin(gama)/c = 1/2R

Law of cosines: a*a=b*b + c*c- 2bc cos(alpha)

Law of tangents: (a+b)/(a-b) = (tan(alpha+beta)/2.0) / (tan(alpha-beta)/2.0)

Area of triangle given all medians m1, m2, m3 : 4/3*Area(m1, m2, m3)

Median Mc of a triangle with length a, b, c: Mc=1/2*sqrt(2*a*a+2*b*b-c*c)

Sidelength 'c' of a triangle with medians Ma, Mb, Mc :

c = 2/3*sqrt(2*Ma*Ma+2*Mb*Mb-Mc*Mc)

= sqrt(2*(b*b+a*a)-4Mc*Mc)

= sqrt(b*b/2-a*a+2*Mb*Mb)

----------Conic / Cylender-------

Cylinderarea: A = PI*r*r*h

Conic area: A = 1/3 * PI*r*r*h

Frustum area: fA = PI*h/3 * (R*R + R*r + r*r)

------- Trapezoid -------

Given Parallel side lengths a, b and height h between them,

Area = (a+b)/2*h

Given four side lengths a, b, c, d where a and b are parallel,

Area = (a+b)/abs(a-b)*sqrt((s-a)(s-b)(s-b-c)(s-b-d))

----------Line related geometry----------

- l1 and l2 having the slopes m1 and m2 , respectively.
- If two lines l1 and l2 are parallel , then m1 = m2.
- If two lines l1 and l2 are perpendicular , then m1 × m2 = –1 .
- The angle θ from l1 to l2 is tanθ = (m2 – m) / (1 + m1 * m2)
- $\frac{|ax_1 + by_1 + c|}{\sqrt{a^2 + b^2}}$ The distance of point (x1,y1) from the line ax + by + c = 0 is ⇒
- If ax+by+c=0 with b>0 is the equation of the line l, the P(x1,y1)lies:
  - **(1)** Above the line l if ax1 + by1 + c > 0
  - **(2)** Below the line l i f ax1 + by1 + c < 0
  - **(3)** On the line

  l if ax1 + by1 + c = 0
- Three lines a1x + b1y + c1 = 0, a2x + b2y + c2 = 0, a3x +b3y + c3 = 0 areconcurrent (সহগামী) if

$$\begin{vmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{vmatrix} = 0$$

```
// Direction vectors
int dRow[] = { -1, 0, 1, 0 };
int dCol[] = { 0, 1, 0, -1 };
bool isValid(bool vis[][COL], int row, int col)
{
   if (row < 0 || col < 0   || row >= ROW ||
col >= COL)
      return false;
   if (vis[row][col])   return false;
   return true;
}
for (int i = 0; i < 4; i++)
{
   int adjx = x + dRow[i];
   int adjy = y + dCol[i];
   if (isValid(vis, adjx, adjy)) {
      q.push({ adjx, adjy });
      vis[adjx][adjy] = true;
   }
}
```

PBDS:

```
#include <bits/stdc++.h>
#include<ext/pb_ds/assoc_container.hpp>
#include<ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
using namespace std;

template <typename T> using pbds = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
template <typename T> using pbds = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;

int main()
{
        order_of_key(val)..gives int.
        find_by_order(pos).//gives iterator
        return 0;
}
```

Algorithm :

BFS:

```
#include <bits/stdc++.h>
using namespace std;
vector<int> v[1005]; bool vis[1005];
void bfs(int src)
{
   queue<int> q;
   q.push(src);
   vis[src] = true;
   while (!q.empty())
   {
      int par = q.front();
      q.pop();
      cout << par << endl;
      for (int child : v[par])
      {
         if (vis[child] == false)
         {
            q.push(child);
            vis[child] = true;
         }
      }
   }
}
   memset(vis, false, sizeof(vis));
   bfs(src);
   return 0;
}
```

DFS:

```
const int N = 1e5 + 5;
vector<int> v[N];bool vis[N];
void dfs(int src)
{
   cout << src << endl;
   vis[src] = true;
   for (int child : v[src])
   {
      if (vis[child] == false)
         dfs(child);
   }
}
```

#SUBLIME CUSTOM BUILD

```
{
"cmd" : ["g++ -std=c++14 $file_name -o $file_base_name && timeout 4s
./$file_base_name<inputf.in>outputf.in"],
"selector" : "source.c",
"shell": true,
"working_dir" : "$file_path"
}
```

#define Fast

```
ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
```

DSU :

```
int dsu_find(int node)
{
   if (parent[node] == -1)
      return node;

   int leader = dsu_find(parent[node]);

   parent[node] = leader;
   return leader;
}
```

```
void dsu_initialize(int n)
{
   for (int i = 0; i < n; i++)
   {
      parent[i] = -1;
      group_size[i] = 1;
   }
}
```

```
void dsu_union_by_size(int node1, int node2)
{
   int leaderA = dsu_find(node1);
   int leaderB = dsu_find(node2);
   if (group_size[leaderA] > group_size[leaderB])
   {
      parent[leaderB] = leaderA;
      group_size[leaderA] += group_size[leaderB];
   }
   else
   {
      parent[leaderA] = leaderB;
      group_size[leaderB] += group_size[leaderA];
   }
```

Bellmanford :

```
#include <bits/stdc++.h>
using namespace std;
class Edge
{
public:
   int u, v, c;
   Edge(int u, int v, int c)
   {
      this->u = u;
      this->v = v;
      this->c = c;
   }
};
```

```
const int N = 1e5 + 5;
int dis[N];
int main()
{
   int n, e;
   cin >> n >> e;
   vector<Edge> EdgeList;
   while (e--)
   {
      int u, v, c;
      cin >> u >> v >> c;
      EdgeList.push_back(Edge(u, v, c));
   }
   for (int i = 0; i < n; i++)
   {
      dis[i] = INT_MAX;
   }
```

```
   dis[0] = 0;
   for (int i = 1; i <= n - 1; i++)
   {
      for (Edge ed : EdgeList)
      {
         int u, v, c;
         u = ed.u;
         v = ed.v;
         c = ed.c;
         if ( dis[u] < INT_MAX  && dis[u] + c < dis[v] )
         {
            dis[v] = dis[u] + c;
         }
      }
   }
   for (int i = 0; i < n; i++)
      cout << i << " -> " << dis[i] << endl;
   return 0;
}
```

Flloyed Warshall :

```
#include <bits/stdc++.h>
#define ll long long int
using namespace std;
int main()
{
   ll n, e;
   cin >> n >> e;
   ll adj[n][n];
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
      {
         adj[i][j] = INT_MAX;
         if (i == j)
            adj[i][j] = 0;
      }
   }
```

```
   while (e--)
   {
      int a, b, c;
      cin >> a >> b >> c;
      adj[a][b] = c;
   }
   for (int k = 0; k < n; k++)
   {
      for (int i = 0; i < n; i++)
      {
         for (int j = 0; j < n; j++)
         {
            if (adj[i][k] + adj[k][j] < adj[i][j])
            {
               adj[i][j] = adj[i][k] + adj[k][j];
            }
         }
      }
   }
```

```
   // cout << "AFTER" << endl;
      // for (int i = 0; i < n; i++)
      // {
      //    for (int j = 0; j < n; j++)
      //    {
      //       if (adj[i][j] == INT_MAX)
      //          cout << "INF ";
      //       else
      //          cout << adj[i][j] << " ";
      //    }
      //    cout << endl;// }
      for (int i = 0; i < n; i++)
      {
         if (adj[i][i] < 0)
         {
            cout << "Cycle detected" << endl;
            break;} }
      return 0;
   }
```

Merge Sort :

```
void merge(int a[],int l,int m,int r)
{
   int leftSize = m-l+1;
   int rightSize = r-m;
   int L[leftSize],R[rightSize];
   int k=0;
   for(int i=l;i<=m;i++)
   {
      L[k]=a[i];
      k++;
   }
   k=0;
   for(int i=m+1;i<=r;i++)
   {
      R[k]=a[i];
      k++;
   }
   int i=0,j=0; int cur=l;
   while(i<leftSize && j<rightSize)
   {
      if(L[i] <= R[j])
      { a[cur]=L[i];  i++; }
      else
      {a[cur]=R[j];j++; }
      cur++;
   }
   while(i<leftSize)
   {a[cur]=L[i]; i++;cur++; }
   while(j<rightSize)
   {a[cur]=R[j];  j++;cur++;}}
```

```
void merge_sort(int a[],int l,int r)
{
   if(l<r)
   {
      int mid=(l+r)/2;
      merge_sort(a,l,mid);
      merge_sort(a,mid+1,r);
      merge(a,l,mid,r);
   }
}
```

```
int main()
{
   int n;
   cin>>n;
   int a[n];
   for(int i=0;i<n;i++)
   {
      cin>>a[i];
   }
   merge_sort(a,0,n-1);
   // for(int i=0;i<n;i++)
   // {
   //    cout<<a[i]<<" ";
   // }
   return 0;
}
```

DP :

TopDoen Approach :

```
int dp[1005][1005];
bool subset_sum(int n, int a[], int s)
{
   if (n == 0)
   {
      if (s == 0)
         return true;
      else
         return false;
   }
   if (dp[n][s] != -1)
      return dp[n][s];
   if (a[n - 1] <= s)
   {
      bool op1 = subset_sum(n - 1, a, s - a[n - 1]);
      bool op2 = subset_sum(n - 1, a, s);
      return dp[n][s] = op1 || op2;
   }
   else
   {
      return dp[n][s] = subset_sum(n - 1, a, s);
   }
}
```

BottomUp Approach :

```
bool dp[n + 1][s + 1];
   dp[0][0] = true;
   for (int i = 1; i <= s; i++)
   {
      dp[0][i] = false;
   }
   for (int i = 1; i <= n; i++)
   {
      for (int j = 0; j <= s; j++)
      {
         if (a[i - 1] <= j)
         {
            dp[i][j] = dp[i - 1][j - a[i - 1]] || dp[i - 1][j];
         }
         else
         {
            dp[i][j] = dp[i - 1][j];
         }
      }
   }
```

Segment tree :

```
const int MAX_N = 200005;
array<int, MAX_N> arr;
long long tree[4 * MAX_N];


void build_tree(int st, int en, int node)
{
   if (st == en)
   {
      tree[node] = arr[st];
      return;
   }
   int mid = (st+en)/2;

   build_tree(st , mid , 2 * node);
   build_tree(mid+1 , en, 2 * node + 1);

   tree[node] = tree[2 * node] + tree[2 * node + 1];

}

int get_range_sum( int st , int en , int node , int L , int R)
{
   /// completely out of range!
   if (R < st || en < L) return 0;
   /// completely within range!
   if (L <= st && en <= R)
   {
      return tree[node];
   }
   int mid = (st + en) /2;
   auto left_cnt = get_range_sum(st, mid, node + node, L, R);

   auto right_cnt = get_range_sum(mid + 1, en, node + node + 1, L, R);

   return left_cnt + right_cnt;
}



void update(int st, int en, int node, int idx, int val)
{
   /// completely out of range!
   if (idx < st || en < idx) return;
   /// completely within range!
   if (st == en && st == idx)
   {
      tree[node] = val;
      return;
   }
   int m = st + (en - st) / 2;
   update(st, m, node + node, idx, val);
   update(m + 1, en, node + node + 1, idx, val);
   tree[node] = tree[2 * node] + tree[(2 * node) + 1];
}
```

```
int main()
{
   int n, q;
   scanf("%d %d", &n, &q);
   for (int i = 0; i < n; i++)
   {
      scanf("%d", &arr[i]);
   }
   build_tree(0, n - 1, 1);
   while(q--)
   {
      int type, l, r;
      scanf("%d %d %d", &type, &l, &r);
      if (type == 2)
      {
         printf("%ld\n", get_range_sum(0, n - 1, 1, l - 1, r - 1));
      }
      else
      {
         update(0, n - 1, 1, l - 1, r);
      }
   }
   return 0;
}
```

```
//dijsktra  algorithm

typedef pair<ll, int> pll;
vector<ll> dist;

void dijkstra(int start, int n, vector<vector<pll>>& adj)
{
   dist.assign(n + 1, 1e18);
   priority_queue<pll, vector<pll>, greater<pll>> pq;
   dist[start] = 0; pq.push({0, start});
   while (!pq.empty()) {
      auto [d, u] = pq.top(); pq.pop();
      if (d > dist[u]) continue;
      for (auto [w, v] : adj[u]) {
         if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            pq.push({dist[v], v});
         }
      }
   }
}
```

```
//sparse_tabel
//min, max , sum , gcd , lcm , or , and

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 200005;
const int K = 20;

int st[MAXN][K];
int lg[MAXN];
void build(vector<int>& a, int n)  //(nlogn)
{
   lg[1] = 0;
   for (int i = 2; i <= n; i++) {
      lg[i] = lg[i / 2] + 1;
   }
   for (int i = 0; i < n; i++) {
      st[i][0] = a[i];
   }
   for (int j = 1; j < K; j++) {
      for (int i = 0; i + (1 << j) <= n; i++) {
      st[i][j] = max(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
      }
   }
}

int query(int L, int R) //o(1)
{
   int j = lg[R - L + 1];
   return max(st[L][j], st[R - (1 << j) + 1][j]);
}

int main() {
   ios::sync_with_stdio(false);
   cin.tie(0);

   int n;
   cin >> n;

   vector<int> a(n);
   for (int i = 0; i < n; i++) cin >> a[i];

   build(a, n);

   int q;
   cin >> q;

   while (q--)
   {
      int L, R;
      cin >> L >> R;

      //  reminder zero based l ,r search so must be
convert the range in zero based

      cout << L << " " << R << query(L, R) << endl;
   }

   return 0;
}
```

1) priority_queue<int> pq;
2) priority_queue<int,vector<int>,greater<int>> pq;

3) for(auto it=mp.rbegin();it!=mp.rend();it++){}

4) prefix xor : px[i]=px[i-1]^a[i];
   subarray_xor=px[r]^px[l-1];
5) set<int> s(all(a));
   int mex=0;
   while(s.count(mex)) mex++;
6) bitmask dp:

```
   for (int mask = 0; mask < (1 << n); mask++) {
  for (int i = 0; i < n; i++) {
     if (!(mask >> i & 1)) {
     }
  }
}
```

7) cout << fixed << setprecision(10) << n << endl;
8) v.erase(unique(all(v)), v.end());
9) __builtin_popcountll(n): Total number of set bits (1s).
10) __builtin_clzll(n): Number of leading zeros (helps find highest set bit).
11) __builtin_ctzll(n): Number of trailing zeros.
12) long long lcm(long long a, long long b) { if (a == 0 || b == 0) return 0; return (a / __gcd(a, b)) * b;

---

100 prime number :

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97.

Prime Count(10-10^26) :

4|25|168|1,229|9,592|78,498|664,579|5,761,455|50,847,534|455,052,511
|4,118,054,813|37,607,912,018|346,065,536,839|3,204,941,750,802|
29,844,570,422,669|279,238,341,033,925|2,623,557,157,654,233|
24,739,954,287,740,860|234,057,667,276,344,607|2,220,819,602,560,918,840|
21,127,269,486,018,731,928|201,467,286,689,315,906,290|
1,925,320,391,606,803,968,923|18,435,599,767,349,200,867,866|
176,846,309,399,143,769,411,680

Template:

```
#include <bits/stdc++.h>
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/pb_ds/assoc_container.hpp>
using namespace std;
using namespace __gnu_pbds;

typedef tree<int, null_type, less<int>, rb_tree_tag,
tree_order_statistics_node_update > pbds;
// find_by_order, order_by_key
#define PI 3.14159265358979323846

#define all(x) x.begin(), x.end()
#define rall(x) x.rbegin(), x.rend()
#define sortall(x) sort(all(x))
#define sortrall(x) sort(rall(x))

#define printv(v) for(auto x : v) cout << x << " "; cout << "\n"
#define printm(m) for(auto x : m) cout << x.F sp x.S << "\n"

// Makes unique array
#define make_unique(x) sortall(x); (x).resize(unique(all(x)) - (x).begin())
// number to string for 32 bit
#define numtobin(n) bitset<32>(n).to_string()
// string to number
#define bintoint(bin_str) stoi(bin_str, nullptr, 2)

template<typename T>
void debug(T x) { cerr << x << endl; }

template<typename T, typename... Args>
void debug(T x, Args... args) { cerr << x << " "; debug(args...); }


void fraction() {
   cout.unsetf(ios::floatfield);
   cout.precision(10);
   cout.setf(ios::fixed,ios::floatfield);
}
```

Prefix Sum 2D:

```
const int N = 1005;
int a[N][N], pref[N][N];

for (int i = 1; i <= n; i++) {
   for (int j = 1; j <= m; j++) {
      pref[i][j] = pref[i - 1][j] + pref[i][j - 1] - pref[i - 1][j - 1] + a[i][j];
   }
}

int q; cin >> q;
while (q--) {
   int x1, y1, x2, y2; cin >> x1 >> y1 >> x2 >> y2;
   int ans = 0;
   ans = pref[x2][y2] - pref[x1 - 1][y2] - pref[x2][y1 - 1] + pref[x1 - 1][y1 - 1];
   cout << ans << '\n';
}
```

```
// Multiply caring overflow
ll mulmod(ll a, ll b, ll m = MOD) {
ll r=0;
   for (a %= m; b; b>>=1, a=(a*2)%m) if (b&1) r=(r+a)%m;
   return r;
}

ll gcd(ll a, ll b) { return b ? gcd(b, a%b) : a; }
ll lcm(ll a, ll b) { return a/gcd(a, b)*b; }

// Fast exponential
ll fexp(ll a, ll b, ll m = MOD) {
   ll r=1;
   for (a %= m; b; b>>=1, a=(a*a)%m) if (b&1) r=(r*a)%m;
   return r;
}
```

Sum of all Divisors (SOD):

```
// first generate prime numbers
ll SOD( ll n ) {
   ll sod = 1;
   for ( auto p : primes ) {
      if( 1LL * p * p > n ) break;
      if( n % p == 0 ) {
         ll sum = 1;
         ll a = 1;
         while ( n % p == 0 ) {
            a *= p;
            sum += a;
            n /= p;
         }
         sod *= sum;
      }
   }
   if(n > 1) sod *= (n + 1);
   return sod;
}
```

Euler Phi:

```
// first generate prime numbers
int PHI(ll n)
{
   ll res = n;
   for(auto p : primes)
    {
      if(1ll * p * p > n) break;
      if(n % p == 0) {
         res /= p, res *= p-1;
         while(n % p == 0) n /= p;
      }
   }
   if(n > 1) {
      res /= n, res *= n - 1;
   }
   return res;
}
```

Knight Moves inInfinity Grid:

```
// Minimum number of knight moves from (x,y) to
// (0,0) in non-negative infinite chessboard

ll knight_move(ll x, ll y)
{
  ll cnt = max({(x + 1) / 2, (y + 1) / 2, (x + y + 2) / 3});
  while((cnt % 2) != (x + y) % 2) cnt++;
  if(x == 1 && !y) return 3;
  if(y == 1 && !x) return 3;
  if(x == y && x == 2) return 4;
  return cnt;
}
```

```
//0/1 knapsack
// W = max capacity, n = number of items
// values[] and weights[] are 0-indexed
long long knapsack(int W, int n, vector<int>& weights, vector<int>&
values) {
   // 1D array saves memory: dp[w] is max value for capacity w
   vector<long long> dp(W + 1, 0);

   for (int i = 0; i < n; i++) {
      // IMPORTANT: Iterate BACKWARDS to ensure each item is used only
once
      for (int w = W; w >= weights[i]; w--) {
         dp[w] = max(dp[w], dp[w - weights[i]] + values[i]);
      }
   }
   return dp[W];
}

//Longest Increasing Subsequence (LIS) - $O(Nlog N)
int getLIS(vector<int>& nums) {
   if (nums.empty()) return 0;

   vector<int> tails;
   for (int x : nums) {
      // Use lower_bound for strictly increasing
      // Use upper_bound for non-decreasing (allows equal elements)
      auto it = lower_bound(tails.begin(), tails.end(), x);

      if (it == tails.end()) {
         tails.push_back(x); // x is larger than any element in tails
      } else {
         *it = x; // Replace the first element >= x with x
      }
   }
   return tails.size();
}

//Bitmask DP TemplateUse
//Case:  $N$ is very small (N<=20).
int n;
int cost[20][20];
int memo[1 << 20][20];

int solve(int mask, int last) {
   if (mask == (1 << n) - 1) return 0; // All visited
   if (memo[mask][last] != -1) return memo[mask][last];

   int ans = 1e9;
   for (int i = 0; i < n; i++) {
      if (!(mask & (1 << i))) { // If i-th bit is not set (not visited)
         ans = min(ans, solve(mask | (1 << i), i) + cost[last][i]);
      }
   }
   return memo[mask][last] = ans;
}
```

Prime Generator:

```
const int mx = 1e6+123;
bitset<mx> isPrime;
vector<int> primes;
int cntPrime[mx];

void primeGen(){
   for ( int i = 3; i < mx; i += 2 ) isPrime[i] = 1;

   int sq = sqrt(mx);
   for ( int i = 3; i <= sq; i += 2 ) {
      if(isPrime[i]) {
         for ( int j = i*i; j < mx; j += i ) {
            isPrime[j] = 0;
         }
      }
   }

   isPrime[2] = 1;
   primes.push_back(2);

   for ( int i = 3; i < mx; i += 2 ) {
      if(isPrime[i] == 1) {
         primes.push_back(i);
      }
   }
}
```

ALL FORMULA :

1. $1+2+3+\cdots+n=\frac{n(n+1)}{2}$
2. $2+4+6+\cdots+2n=n(n+1)$
3. $1+4+7+\cdots+(3n-2)=\frac{n(3n-1)}{2}$
4. $5+8+11+\cdots+(3n+2)=\frac{n(3n+7)}{2}$
5. $1^2+2^2+3^2+\cdots+n^2=\frac{n(n+1)(2n+1)}{6}$
6. $1\cdot 2+2\cdot 3+3\cdot 4+\cdots+n(n+1)=\frac{n(n+1)(n+2)}{3}$
7. $1\cdot 3+2\cdot 4+3\cdot 5+\cdots+n(n+2)=\frac{n(n+1)(2n+7)}{6}$
8. $1^3+2^3+3^3+\cdots+n^3=\frac{n^2(n+1)^2}{4}$
9. $1^3+3^3+5^3+\cdots+(2n-1)^3=n^2(2n^2-1)$
10. $2^3+4^3+6^3+\cdots+(2n)^3=2n^2(n+1)^2$
11. $\frac{1}{1\cdot 2}+\frac{1}{2\cdot 3}+\frac{1}{3\cdot 4}+\cdots+\frac{1}{n(n+1)}=\frac{n}{n+1}$
12. $1\cdot 2+2\cdot 2^2+3\cdot 2^3+4\cdot 2^4+\cdots+n\cdot 2^n=2[1+(n-1)2^n]$
13. $1+2+2^2+\cdots+2^{n-1}=2^n-1$

XOR 1 to N:

$$\text{XOR}(1 \text{ to } N) = \begin{cases} N & \text{if } N \equiv 0 \pmod 4 \\ 1 & \text{if } N \equiv 1 \pmod 4 \\ N+1 & \text{if } N \equiv 2 \pmod 4 \\ 0 & \text{if } N \equiv 3 \pmod 4 \end{cases}$$

Number theory all :

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const int MOD = 1e9 + 7;
// Covers up to 1 million. Adjust based on memory.
const int MAX = 1000005;

struct NumberTheory {
   int spf[MAX];
   ll fact[MAX], invFact[MAX];

   NumberTheory() {
      // 1. Sieve initialization
      for (int i = 0; i < MAX; i++) spf[i] = i;
      for (int i = 2; i * i < MAX; i++) {
         if (spf[i] == i) {
            for (int j = i * i; j < MAX; j += i)
               if (spf[j] == j) spf[j] = i;
         }
      }

      // 2. Precompute Factorials for nCr
      fact[0] = 1;
      for (int i = 1; i < MAX; i++) fact[i] = (fact[i - 1] * i) % MOD;
      invFact[MAX - 1] = power(fact[MAX - 1], MOD - 2);
      for (int i = MAX - 2; i >= 0; i--)
         invFact[i] = (invFact[i + 1] * (i + 1)) % MOD;
   }

   // Binary Exponentiation: a^b % MOD
   ll power(ll base, ll exp) {
      ll res = 1; base %= MOD;
      while (exp > 0) {
         if (exp % 2 == 1) res = (res * base) % MOD;
         base = (base * base) % MOD;
         exp /= 2;
      }
      return res;
   }

   // Modular Inverse: (1/n) % MOD (Only works if MOD is prime)
   ll modInverse(ll n) { return power(n, MOD - 2); }

   // nCr: O(1) combinations
   ll nCr(int n, int r) {
      if (r < 0 || r > n) return 0;
      return (((fact[n] * invFact[r]) % MOD) * invFact[n - r]) % MOD;
   }

   // Factorization: Returns {prime, power} pairs. O(log N)
   vector<pair<ll, int>> factorize(ll n) {
      vector<pair<ll, int>> factors;
      // If n is within sieve range, use fast SPF
      if (n < MAX) {
         while (n > 1) {
            int p = spf[n], count = 0;
            while (n % p == 0) { n /= p; count++; }
            factors.push_back({p, count});
         }
      } else { // If n is large, use O(sqrt N) trial division
         for (ll i = 2; i * i <= n; i++) {
            if (n % i == 0) {
               int cnt = 0;
               while (n % i == 0) { n /= i; cnt++; }
               factors.push_back({i, cnt});
            }
         }
         if (n > 1) factors.push_back({n, 1});
      }
      return factors;
   }

// Euler Phi: Count numbers 1 to n coprime to n. O(sqrt N)
   ll phi(ll n) {
      ll res = n;
      for (ll i = 2; i * i <= n; i++) {
         if (n % i == 0) {
            while (n % i == 0) n /= i;
            res -= res / i;
         }
      }
      if (n > 1) res -= res / n;
      return res;
   }

  // Extended GCD: finds x, y for ax + by = gcd(a, b)
   ll extGCD(ll a, ll b, ll &x, ll &y) {
      if (b == 0) { x = 1; y = 0; return a; }
      ll x1, y1, d = extGCD(b, a % b, x1, y1);
      x = y1; y = x1 - y1 * (a / b);
      return d;
   }
};

// GLOBAL: Initialize once
NumberTheory nt;

Int main()
{
cout << "             nCr: " << nt.nCr(10, 3) << "\n";
{
```